

Extending RANsacked for NAS-Aware and Stateful Fuzzing of 5G Control-Plane Interfaces

Omkar Chavan (CS24MTECH12013)

Abstract

This project investigates robustness testing of 5G control-plane software through trace-driven, NAS-aware fuzzing of Open5GS. The focus is on NGAP-carried NAS messages processed early or at security-sensitive stages of the registration procedure. Instead of relying on naive random mutation, the project models selected NAS messages, fields, mutation families, and operators, then executes supported live mutations through a custom SCTP NGAP proxy placed between UERANSIM and Open5GS. The workflow also includes campaign planning, history tracking, log-based classification, and report generation. The final freeze-state evaluation covered 5 modeled NAS message types, 24 summarized mutation families, and 97 recorded campaign observations. Results show that malformed-but-structured NAS inputs do not collapse into a single behavior class. Depending on the message, field, and procedure stage, observed outcomes included early semantic rejection, deeper decoder failure, timeout/retry behavior, and malformed-but-accepted behavior. The strongest overall findings came from *Registration Request :: mobile-identity value corruption*, while *Security Mode Complete :: wrong-state delivery* provided the clearest state-aware results. The project demonstrates that NAS-aware and selected state-aware fuzzing can provide meaningful insights beyond simple packet mutation while remaining reproducible in a software-only Open5GS + UERANSIM testbed.

1 Introduction / Project Recap

5G control-plane interfaces such as NGAP and NAS are security-critical because they process externally influenced inputs very early in the protocol flow [1, 2]. In practice, many malformed inputs are rejected too early when fuzzing is purely random or packet-oriented, which motivates structure-aware mutation that preserves enough protocol shape to exercise deeper parsing, validation, and procedure logic [1, 2]. The goal of this project was to build a working trace-driven fuzzing workflow for Open5GS that supports NAS-aware structured mutation, selected state-aware experimentation, and reportable campaign execution. The final implementation focused on a software-only testbed using Open5GS [3] and UERANSIM [4], with a custom SCTP NGAP proxy used as the live mutation point. The overall workflow is implementation-oriented and complements generic fuzzing paradigms such as AFL++ [5] and LibFuzzer [6], while remaining targeted to realistic 5G control-plane experimentation. The implementation snapshot and supporting artifacts are maintained in the project repository [7].

2 Related Work and Positioning

Prior telecom fuzzing work has shown that structure-aware mutation is more effective than naive malformed-packet generation for exercising control-plane logic [1, 2]. RANSacked is an important motivating example because it demonstrates the value of domain-informed fuzzing on telecom signaling interfaces [1]. A closely related recent direction is the ITASEC 2025 work on optimizing 5G protocol fuzzing through structure-aware techniques and feedback integration [2]. That work validates the importance of meaningful, decodable inputs and deeper state reachability [2]. Generic fuzzing engines such as AFL++ [5] and LibFuzzer [6] provide strong coverage-guided backends, but they do not directly provide telecom-specific message modeling, realistic procedure context, or Open5GS-oriented campaign workflows. This project is positioned between those two ends: it is not a generic fuzzing engine, but a concrete NAS-aware Open5GS campaign prototype built around Open5GS [3], with explicit mutation-family modeling, live proxy execution, classification, history tracking, and report generation. The implementation and supporting artifacts are maintained in the project repository [7].

3 System Model / Threat Model

The system under study is the Open5GS AMF [3], exercised through a software-only UERANSIM-based testbed [4]. UERANSIM provides the UE and gNB components needed to generate realistic 5G control-plane traffic [4], while a custom SCTP NGAP proxy sits between the gNB and the AMF to inspect and mutate selected NAS payloads. The primary focus is NGAP-carried NAS messages that appear early in registration or at selected later control-plane stages such as authentication and security mode completion [1, 2]. The threat model assumes either a malicious UE or a compromised forwarding point capable of injecting malformed but protocol-shaped control-plane traffic toward the AMF, consistent with the broader control-plane fuzzing perspective in prior telecom and 5G protocol fuzzing work [1, 2]. The project does not model over-the-air attacks, SDR-based experimentation, or lower-layer radio abuse. Instead, it focuses on robustness of NAS and control-plane handling under structured malformed inputs in a controlled software environment built around Open5GS [3] and UERANSIM [4].

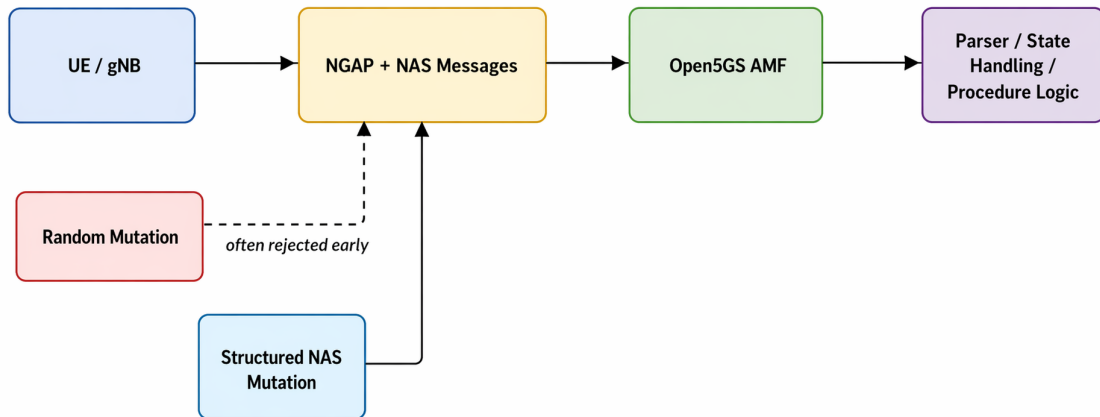


Figure 1. System and threat model for the software-only Open5GS + UERANSIM testbed.

4 System Architecture and Methodology

The overall architecture combines realistic traffic generation with a mutation and campaign-analysis layer. UERANSIM UE and gNB generate normal 5G control-plane traffic [4]. A custom SCTP NGAP proxy is inserted between the gNB and Open5GS AMF [3], where it can either relay traffic transparently or apply one selected mutation to the embedded NAS content. On top of this live path, Python tooling models the NAS mutation space, recommends executable candidates, creates run plans, executes campaigns, classifies outcomes from collected logs, records observations, and exports summaries and reports. The methodology begins with a valid baseline trace or valid-enough live procedure, then introduces one structured mutation at a time, following the broader structure-aware fuzzing rationale emphasized in prior telecom and 5G protocol fuzzing work [1, 2]. The resulting behavior is classified through evidence from AMF, proxy, gNB, and UE logs. This workflow makes the campaign repeatable and suitable for systematic comparison across messages and mutation families, while the implementation and artifacts are maintained in the project repository [7].

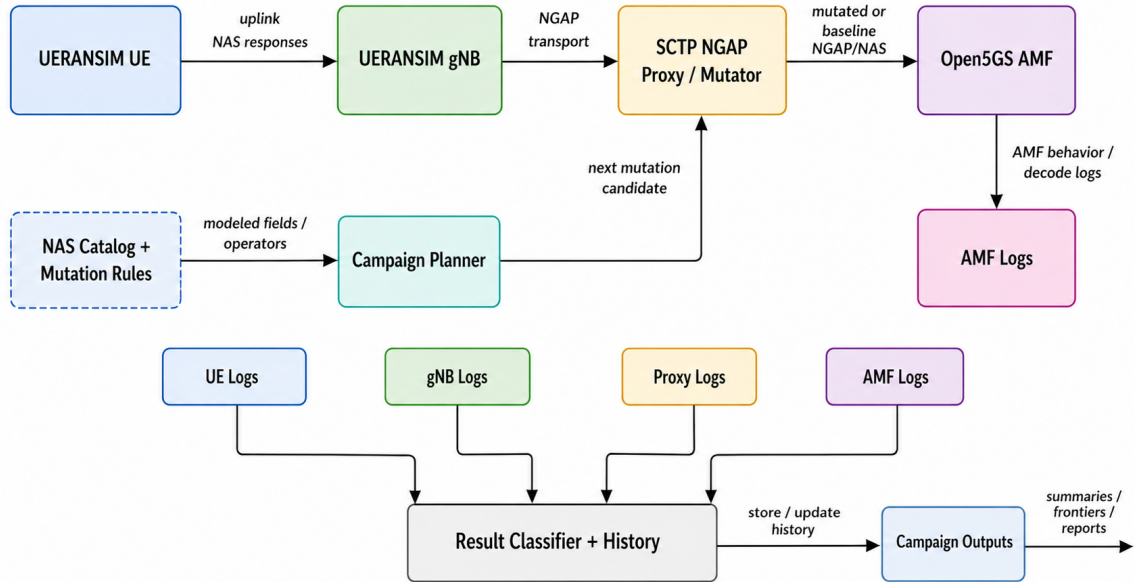


Figure 2. Live execution path and campaign-analysis pipeline used in the final freeze-state evaluation.

5 Core Algorithm / Design Logic

The core logic of the system is a history-aware campaign loop. First, a valid baseline trace or supported live procedure is identified so that mutations remain protocol-grounded, consistent with the structure-aware testing perspective emphasized in prior telecom and 5G protocol fuzzing work [1, 2]. Second, the NAS model enumerates candidate mutation families and operators for the chosen message. Third, the scheduler prioritizes the next frontier using support status, prior history, and family saturation. Fourth, the selected case is executed either live through the proxy or through a simulation-backed path when live execution is not yet available. Fifth, the system collects logs from AMF, proxy, gNB, and UE and classifies the outcome into categories such as `unexpected-accept`, `early-semantic-reject`, `deep-decoder-failure`, or `timeout-retry`.

Finally, the observation is written into campaign history and the family summaries and reports are regenerated. This loop is what turns the project from a mutator into a campaign system built around Open5GS [3], UERANSIM [4], and the project’s own execution/reporting workflow [7].

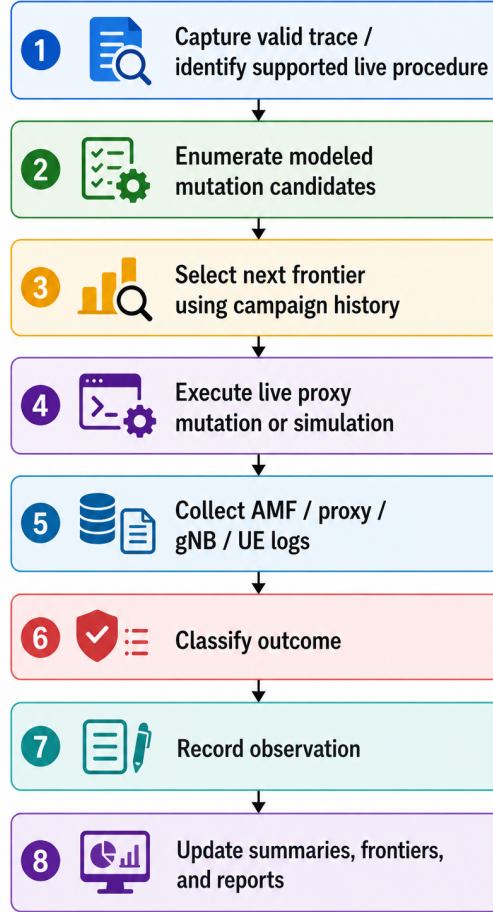


Figure 3. History-aware campaign loop used to select, execute, classify, and record mutation cases.

6 Implementation Details

The implementation uses Open5GS as the target 5G core [3], UERANSIM as the UE/gNB traffic source [4], and a custom SCTP NGAP proxy for live mutation injection. The main software contribution is the Python-based NAS campaign layer, which includes schema-driven NAS message modeling, mutation-family and operator generation, execution-bridge logic, campaign planning, history tracking, classification, and reporting. The strongest early live support was first stabilized on *Registration Request*, after which the same framework was extended to *Identity Response*, *Authentication Response*, *Security Mode Complete*, and *PDU Session Establishment Request*. This implementation direction is aligned with the broader motivation from prior structure-aware telecom and 5G protocol fuzzing work [1, 2], while remaining a concrete Open5GS-oriented campaign prototype rather than a generic fuzzing engine. Challenges included preserving enough structure for live delivery, mapping modeled operators to concrete proxy mutations, maintaining reproducibility across repeated runs, and distinguishing between

shallow semantic rejects and deeper decode failures. The implementation and supporting artifacts are maintained in the project repository [7].

7 Experimental Setup

Experiments were conducted in a software-only setup with Open5GS AMF as the target [3], UERANSIM UE/gNB as the traffic source [4], and the SCTP NGAP proxy as the mutation point. The evaluation used both live proxy-backed mutation and simulation-backed analysis for selected modeled cases. Campaign execution was driven through CLI commands that select a mutation frontier, generate a run plan, execute the run, classify the outcome, record the observation, and summarize the campaign state. Result classes used throughout the evaluation were **unexpected-accept**, **early-semantic-reject**, **deep-decoder-failure**, and **timeout-retry**. As qualitative baselines, the analysis contrasts the approach against simpler packet-level mutation inspired by RANsacked [1] and opaque-byte NAS mutation, both of which generally lack the explicit message- and field-aware structure emphasized in prior structure-aware 5G fuzzing work [1, 2]. Reproducibility is supported through saved histories, logs, run plans, Markdown reports, and a frozen final submission snapshot maintained in the project repository [7].

8 Results and Analysis

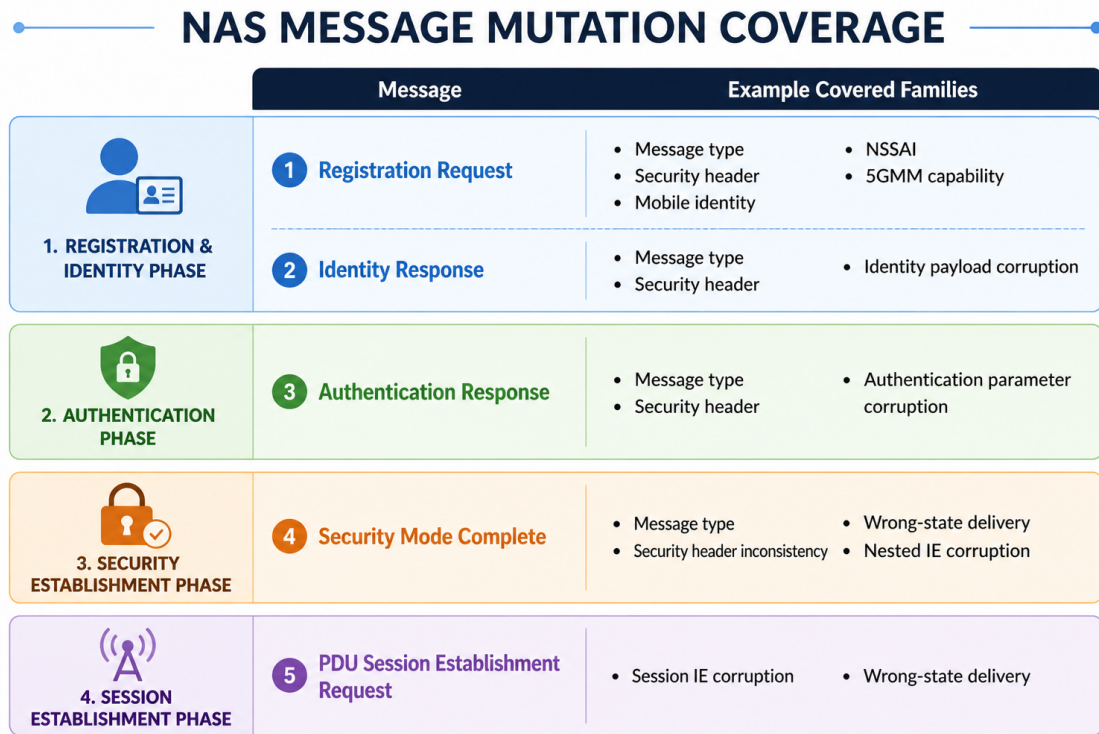
The final freeze-state evaluation covered 5 modeled NAS messages, 24 summarized mutation families, and 97 recorded observations. Results show that structured malformed NAS inputs produced diverse behaviors rather than only uniform rejection, which is consistent with the motivation behind prior structure-aware telecom and 5G protocol fuzzing work [1, 2]. *Registration Request* provided the richest family diversity and the strongest overall finding through *mobile-identity value corruption*, which produced **early-semantic-reject**, **deep-decoder-failure**, and **unexpected-accept** across different operators. *Identity Response :: identity payload corruption* converged to repeated **unexpected-accept**, making it the cleanest end-to-end live demo case. *Authentication Response :: authentication parameter corruption* produced a three-way split across **early-semantic-reject**, **unexpected-accept**, and **timeout-retry**, showing that payload corruption effects depend strongly on how the parameter is malformed. *Security Mode Complete :: wrong-state delivery* produced the clearest explicit state-aware result, with one operator rejected early and another behaving permissively enough to allow registration completion. Overall, the results indicate that the AMF’s response depends on message context, field semantics, and procedure state rather than only on whether the input is malformed in the Open5GS-based evaluation environment [3, 7].

Freeze-State Snapshot

- Modeled NAS messages: 5
- Summarized mutation families: 24
- Recorded campaign observations: 97

Table 1. *High-level results by modeled NAS message.*

Message	Strongest Covered Family	Main Observed Outcome	Final State
Registration Request	Mobile identity / Requested NSSAI / 5GMM capability	Unexpected accept, deep decoder failure, early semantic reject	Strong live coverage
Identity Response	Identity payload corruption	Unexpected accept	Likely saturated
Authentication Response	Authentication parameter corruption	Unexpected accept, early semantic reject, timeout retry	Fully explored
Security Mode Complete	Wrong-state delivery, nested IE corruption	Early semantic reject, unexpected accept, timeout retry	Strong live coverage
PDU Session Establishment Request	Session IE corruption, wrong-state delivery	Mixed live outcomes	Fully explored

**Figure 4.** *NAS mutation coverage*

The detailed mutation-family results are provided in the following supplementary document:: Mutation Families and Main Outcomes.

9 Limitations and Failure Analysis

The main limitation of the current system is the bounded live execution scope. Although the modeled NAS catalog is broader, the implementation is strongest for the families that were promoted into the live proxy-executable path. Some families remain modeled but unexplored because they do not yet have a live execution bridge, such as *Identity Response :: length inconsistency*, *Identity Response :: wrong-state delivery*, and *Authentication Response :: wrong-state delivery*. The project also does not yet provide fully generic execution for any modeled field across all messages. Another limitation is that baseline comparison remains qualitative rather than a full benchmark study with coverage or performance numbers, even though the comparison is motivated by prior structure-aware fuzzing work and generic fuzzing backends such as AFL++ and LibFuzzer [1, 2, 5, 6]. Finally, the results are specific to the current Open5GS [3] + UERANSIM [4] software environment and should not be overgeneralized to all 5G core implementations. The implementation scope and supporting artifacts are documented in the project repository [7].

10 Evaluation Against Objectives

Table 2. *Evaluation against the original project objectives.*

Objective	Outcome	Evidence
Build a trace-driven fuzzing workflow for 5G control-plane testing	Completed	Working Open5GS + UERANSIM + proxy workflow with planning, execution, classification, and reporting
Implement NAS-aware structured mutation inside realistic NGAP/NAS exchanges	Completed	Schema-driven catalog across 5 modeled NAS messages
Support selected procedure-level / state-aware mutation scenarios	Partially completed	Wrong-state delivery implemented for selected Security Mode Complete and PDU Session Establishment Request cases
Evaluate behavior through live execution, history tracking, and reportable analysis	Completed	97 recorded observations, per-message summaries, exported Markdown reports
Compare against simpler mutation baselines	Partially completed	Qualitative comparison against packet-level mutation and opaque-byte NAS mutation

The core project objectives were achieved. The remaining gaps are broader stateful coverage and more comprehensive baseline evaluation, not absence of a working system.

11 Demo Description

The final demo includes two representative live campaign cycles.

- Demo case 1: *Registration Request :: mobile-identity value corruption :: toggle identity type*

bits inconsistently

- Observed result for demo case 1: **unexpected-accept**
- Demo case 2: *Identity Response :: identity payload corruption :: invalid BCD*
- Observed result for demo case 2: **unexpected-accept**

In both cases, the campaign recommends a runnable mutation, executes it through the proxy-backed Open5GS + UERANSIM path, collects logs, classifies the result, records the observation, and updates the campaign summary. The Registration Request case demonstrates a stronger semantic corruption inside the mobile-identity field while preserving the outer Registration Request structure. The Identity Response case demonstrates a payload-level corruption where the tail octet is rewritten to 0x2a while preserving the outer Identity Response shell. In both runs, the malformed but structured messages still allowed the control-plane flow to progress far enough for successful registration, leading to classification as **unexpected-accept**. These two cases were chosen because together they show both the strongest-family style of finding and the cleanest end-to-end live workflow.

The step-by-step live demo workflow is documented in the supplementary document: [FINAL DEMO: Live TWiN Campaign Workflow](#).

12 Work Distribution

This was a solo project, and all technical, experimental, and documentation work was carried out by me. This included environment setup, Open5GS and UERANSIM integration, custom SCTP NGAP proxy development, NAS mutation modeling, campaign tooling, experiment execution, results analysis, slide preparation, and report writing. The GitHub repository history serves as the primary evidence of implementation ownership and development progression.

13 Conclusion

This project built a working trace-driven, NAS-aware, and partially state-aware fuzzing workflow for 5G control-plane testing in Open5GS. The final system combines realistic software-only traffic generation with live mutation injection, campaign planning, classification, and reporting. The freeze-state evaluation covered 5 modeled NAS messages, 24 mutation families, and 97 recorded observations. The main contribution is not a generic fuzzing engine, but a concrete Open5GS-oriented campaign system that demonstrates how structured malformed inputs can reveal diverse control-plane behaviors beyond naive mutation.

14 Future Work

- Promote more modeled families into the live execution path, especially the currently planned-only Identity Response and Authentication Response families.
- Make execution support more generic so that any modeled NAS field can be exercised without family-specific bridge code.

- Extend state-aware experimentation across more procedures and later-message control-plane contexts.
- Broaden protocol coverage to include richer NGAP and 5GSM mutation scenarios.
- Explore direct decoder-level fuzzing of Open5GS internals as a white-box complement to the current system-level approach.
- Investigate AFL++ or LibFuzzer-style instrumentation for deeper parser-focused fuzzing in future work.

References

- [1] RANsacked: A Domain-Informed Approach for Fuzzing LTE and 5G RAN-Core Interfaces.
- [2] Optimizing 5G Protocol Fuzzing: Structure-Aware Techniques and Feedback Integration, ITASEC 2025.
- [3] Open5GS: Open-source 5G Core and EPC implementation.
- [4] UERANSIM: Open-source 5G UE and gNB simulator.
- [5] AFL++: Advanced coverage-guided fuzzing framework.
- [6] LibFuzzer: LLVM in-process coverage-guided fuzzing engine.
- [7] Project repository: <https://github.com/omya3/TWIN-5G-fuzzing/tree/post-midterm-generalization>

15 Declaration on LLM Usage

LLM tools, including OpenAI Codex / ChatGPT, were used as coding, debugging, and writing-support tools during the project. Typical uses included clarifying telecom fuzzing concepts, refining mutation-family documentation, assisting with Python/C debugging, improving report wording, and reorganizing presentation content. All generated suggestions were manually reviewed before use. Code changes were validated through local and remote testing, and experimental claims were accepted only after reproducible behavior was observed in the Open5GS + UERANSIM setup.